
Session 09 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

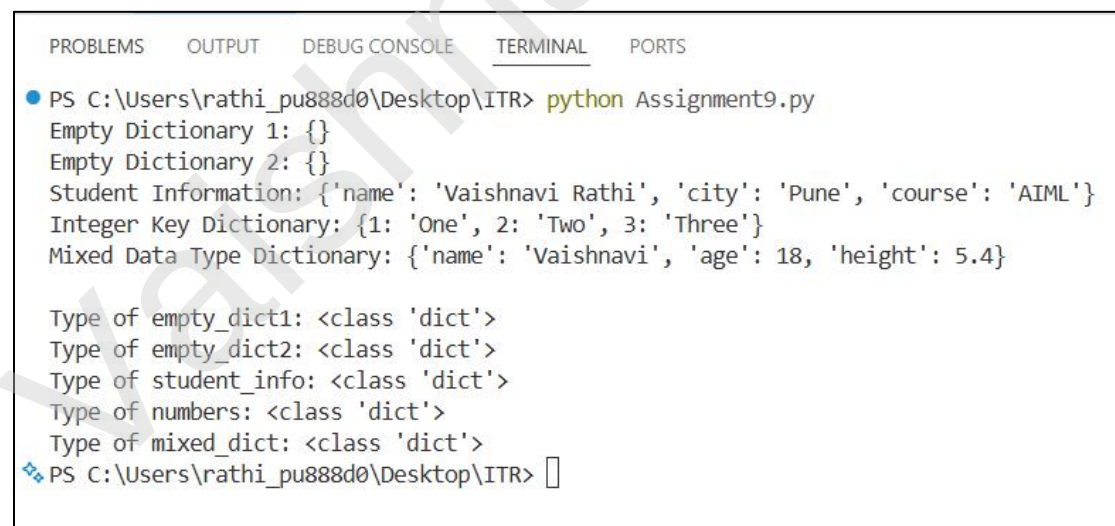
College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

Domain: **AIML (Artificial Intelligence and Machine Learning)**

Q1. Creating Dictionaries

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

● PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Empty Dictionary 1: {}
Empty Dictionary 2: {}
Student Information: {'name': 'Vaishnavi Rathi', 'city': 'Pune', 'course': 'AIML'}
Integer Key Dictionary: {1: 'One', 2: 'Two', 3: 'Three'}
Mixed Data Type Dictionary: {'name': 'Vaishnavi', 'age': 18, 'height': 5.4}

Type of empty_dict1: <class 'dict'>
Type of empty_dict2: <class 'dict'>
Type of student_info: <class 'dict'>
Type of numbers: <class 'dict'>
Type of mixed_dict: <class 'dict'>
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q1. (Creating Dictionaries)

Write a program to create dictionaries using different methods:

- a) An empty dictionary using two different ways.
- b) A dictionary with string keys (name, city, course).
- c) A dictionary with integer keys.
- d) A mixed data type dictionary (string, int, float).

Print each dictionary and its type.

Add comments explaining the syntax.

```
'''
```

```
# a) Creating empty dictionaries using two methods
```

```
empty_dict1 = {}  
empty_dict2 = dict()
```

```
# b) Dictionary with string keys
```

```
student_info = {  
    "name": "Vaishnavi Rath",  
    "city": "Pune",  
    "course": "AIML"  
}
```

```
# c) Dictionary with integer keys
```

```
numbers = {  
    1: "One",  
    2: "Two",  
    3: "Three"  
}
```

```
# d) Mixed data type dictionary
```

```
mixed_dict = {  
    "name": "Vaishnavi",  
    "age": 18,  
    "height": 5.4  
}
```

```
# Print dictionaries
```

```
print("Empty Dictionary 1:", empty_dict1)  
print("Empty Dictionary 2:", empty_dict2)  
print("Student Information:", student_info)  
print("Integer Key Dictionary:", numbers)
```

```
print("Mixed Data Type Dictionary:", mixed_dict)
```

```
print()
```

```
# Print types
```

```
print("Type of empty_dict1:", type(empty_dict1))
```

```
print("Type of empty_dict2:", type(empty_dict2))
```

```
print("Type of student_info:", type(student_info))
```

```
print("Type of numbers:", type(numbers))
```

```
print("Type of mixed_dict:", type(mixed_dict))
```

```
'''
```

```
Explanation in Comments
```

```
# Dictionaries are created using curly braces {}.
```

```
# A dictionary stores data in key:value pairs.
```

```
# Example:
```

```
# "name": "Vaishnavi"
```

```
# Here "name" is the key and "Vaishnavi" is the value.
```

```
# Keys must be unique within a dictionary.
```

```
# An empty dictionary can be created using {} or dict().
```

```
# Dictionaries are mutable, which means values can be added,
```

```
# modified, or removed after creation.
```

```
'''
```

Q2. Accessing and Modifying Elements

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Student Name: Vaishnavi Rathi

Updated Dictionary:
{'name': 'Vaishnavi Rathi', 'age': 18, 'city': 'Pune', 'marks': 92, 'grade': 'A'}
PS C:\Users\rathi_pu888d0\Desktop\ITR> 
```

Program Code:

""" ASSIGNMENT 9 """

'''

Q2. (Accessing and Modifying Elements)

Create the dictionary:

```
student = {
    "name": "YourName",
    "age": 00,
    "city": "YourCity",
    "marks": 00
}
```

Print the value of "name" using square brackets.

Update the "marks" to 92.

Add a new key "grade" with value "A".

Print the updated dictionary.

'''

Creating the dictionary

```
student = {
    "name": "Vaishnavi Rathi",
    "age": 18,
    "city": "Pune",
    "marks": 85
}
```

```
# Accessing the value of "name"
print("Student Name:", student["name"])
```

```
# Updating the marks
student["marks"] = 92
```

```
# Adding a new key-value pair
student["grade"] = "A"
```

```
# Printing the updated dictionary
print("\nUpdated Dictionary:")
print(student)
```

```
'''
```

Explanation in Comments

```
# Values in a dictionary can be accessed using square brackets [].
```

```
# student["name"] returns the value associated with the key "name".
```

```
# Existing values can be modified by assigning a new value to the key.
```

```
# New key-value pairs can be added by using:
```

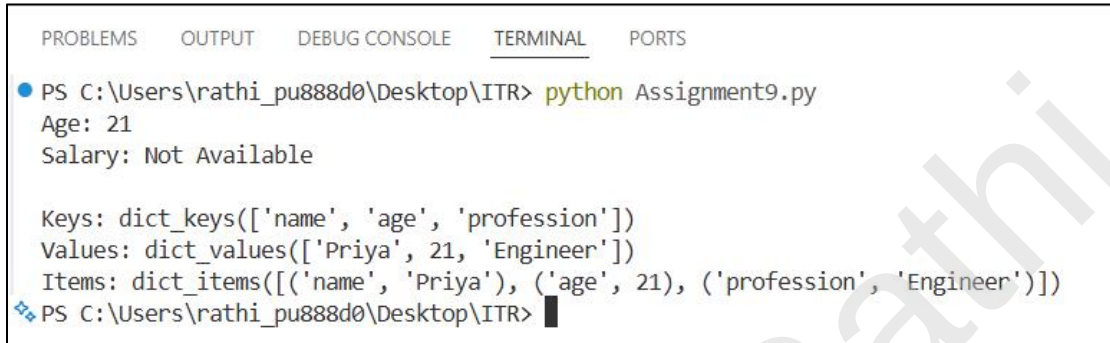
```
# dictionary_name["new_key"] = value
```

```
# Dictionaries are mutable, so their contents can be changed after creation.
```

```
'''
```

Q3. get(), keys(), values(), items()

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Age: 21
Salary: Not Available

Keys: dict_keys(['name', 'age', 'profession'])
Values: dict_values(['Priya', 21, 'Engineer'])
Items: dict_items([('name', 'Priya'), ('age', 21), ('profession', 'Engineer')])
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q3. (get(), keys(), values(), items())

Create a dictionary:

```
person = {
    "name": "Priya",
    "age": 21,
    "profession": "Engineer"
}
```

Demonstrate:

- a) Using get() to access "age" and a non-existing key "salary" (with default value).
 - b) Print all keys using keys().
 - c) Print all values using values().
 - d) Print all key-value pairs using items().
- ```
'''
```

```
Creating the dictionary
```

```
person = {
 "name": "Priya",
 "age": 21,
 "profession": "Engineer"
}
```

```
a) Using get()
print("Age:", person.get("age"))
print("Salary:", person.get("salary", "Not Available"))
```

```
b) Print all keys
print("\nKeys:", person.keys())
```

```
c) Print all values
print("Values:", person.values())
```

```
d) Print all key-value pairs
print("Items:", person.items())
```

```
'''
```

Explanation in Comments

```
get(key) is used to access a value safely.
If the key does not exist, it returns None by default.
```

```
get(key, default_value) returns the specified default value
when the key is not found.
```

```
keys() returns all keys of the dictionary.
```

```
values() returns all values of the dictionary.
```

```
items() returns all key-value pairs as tuple pairs.
```

```
These methods are useful for viewing and accessing
dictionary data efficiently.
```

```
'''
```

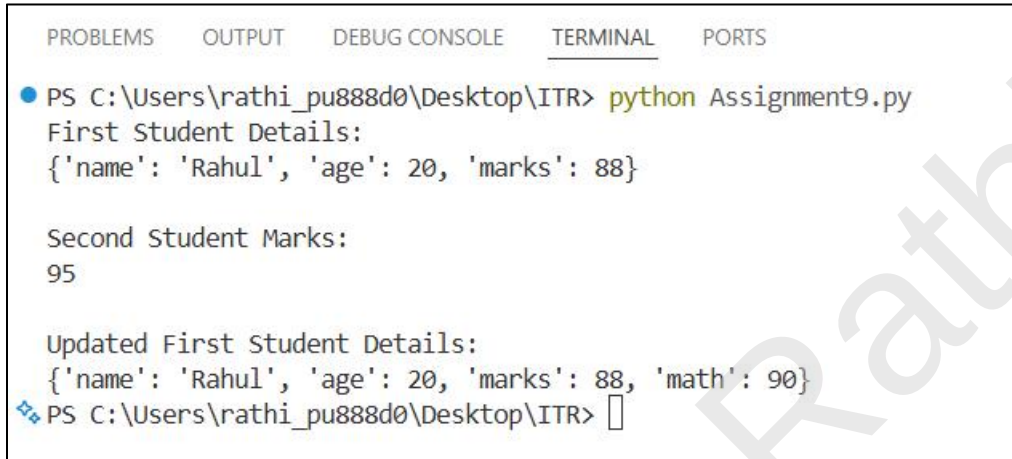
---

---

#### Q4. Nested Dictionary

---

#### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu88d0\Desktop\ITR> python Assignment9.py
First Student Details:
{'name': 'Rahul', 'age': 20, 'marks': 88}

Second Student Marks:
95

Updated First Student Details:
{'name': 'Rahul', 'age': 20, 'marks': 88, 'math': 90}
PS C:\Users\rathi_pu88d0\Desktop\ITR>
```

---

#### Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q4. (Nested Dictionary)

Create a nested dictionary to store information of two students:

```
students = {
 "s1": {"name": "Rahul", "age": 20, "marks": 88},
 "s2": {"name": "Sneha", "age": 21, "marks": 95}
}
```

Print the details of the first student.

Print the marks of the second student.

Add a new subject "math" with marks 90 for the first student.

```
'''
```

```
Creating a nested dictionary
```

```
students = {
 "s1": {"name": "Rahul", "age": 20, "marks": 88},
 "s2": {"name": "Sneha", "age": 21, "marks": 95}
}
```

```
Print details of the first student
```

```
print("First Student Details:")
```

```
print(students["s1"])
```

```
Print marks of the second student
print("\nSecond Student Marks:")
print(students["s2"]["marks"])

Add a new subject "math" for the first student
students["s1"]["math"] = 90

Print updated details of the first student
print("\nUpdated First Student Details:")
print(students["s1"])
```

```
'''
```

Explanation in Comments

```
A nested dictionary is a dictionary inside another dictionary.
```

```
students["s1"] accesses the dictionary of the first student.
```

```
students["s2"]["marks"] accesses the marks value
```

```
from the second student's dictionary.
```

```
New key-value pairs can be added to an inner dictionary
```

```
using the assignment operator.
```

```
students["s1"]["math"] = 90 adds a new subject
```

```
and its marks to the first student's record.
```

```
'''
```

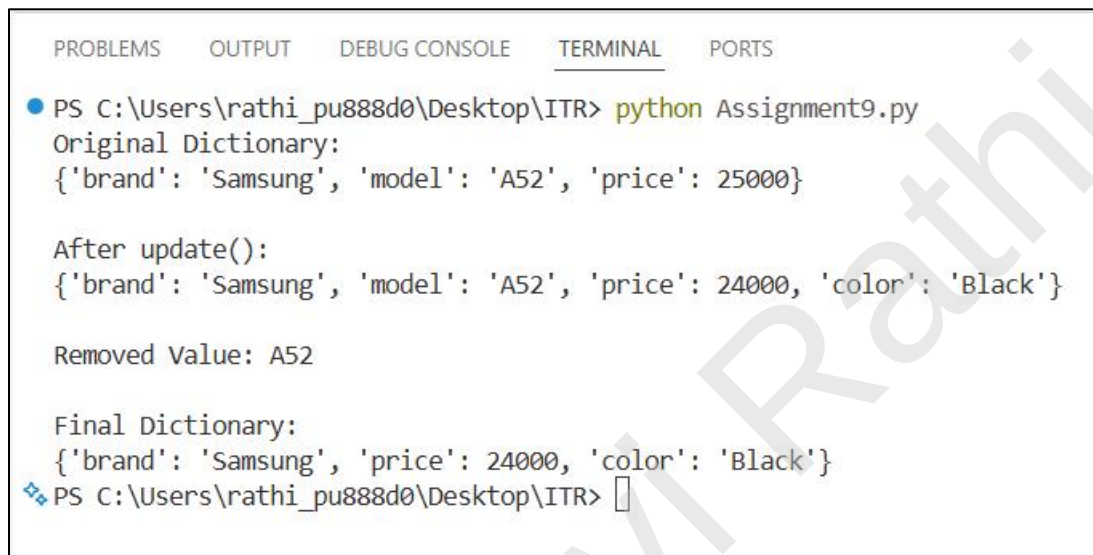
---

---

Q5. update() and pop()

---

**Output Screenshot:**



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Original Dictionary:
{'brand': 'Samsung', 'model': 'A52', 'price': 25000}

After update():
{'brand': 'Samsung', 'model': 'A52', 'price': 24000, 'color': 'Black'}

Removed Value: A52

Final Dictionary:
{'brand': 'Samsung', 'price': 24000, 'color': 'Black'}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

---

**Program Code:**

""" ASSIGNMENT 9 """

'''

Q5. (update() and pop())

Create a dictionary:

info = {"brand": "Samsung", "model": "A52", "price": 25000}

Update it with new information:

{"color": "Black", "price": 24000}

Remove the key "model" using pop() and print the removed value.

Print the final dictionary.

'''

# Creating the dictionary

```
info = {
 "brand": "Samsung",
 "model": "A52",
 "price": 25000
}
```

```
print("Original Dictionary:")
print(info)
```

```
Updating the dictionary
info.update({
 "color": "Black",
 "price": 24000
})
```

```
print("\nAfter update():")
print(info)
```

```
Removing "model" using pop()
removed_value = info.pop("model")
```

```
Print removed value
print("\nRemoved Value:", removed_value)
```

```
Print final dictionary
print("\nFinal Dictionary:")
print(info)
```

```
'''
```

Explanation in Comments

```
update() is used to add new key-value pairs
or modify existing values in a dictionary.
```

```
Here, "color" is added as a new key.
```

```
The value of "price" is updated from 25000 to 24000.
```

```
pop(key) removes the specified key and
returns its associated value.
```

```
The removed value is stored in removed_value.
```

```
'''
```

---

---

Q6. popitem() and clear()

---

### Output Screenshot:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Original Dictionary:
{'name': 'Vaishnavi', 'age': 18, 'city': 'Pune', 'course': 'AIML', 'marks': 92}

First Removed Item: ('marks', 92)
Second Removed Item: ('course', 'AIML')

Dictionary After popitem():
{'name': 'Vaishnavi', 'age': 18, 'city': 'Pune'}

Dictionary After clear():
{}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 9 """

'''
Q6. (popitem() and clear())

Create a dictionary with at least 5 key-value pairs.

Use popitem() twice and print the removed items each time.

Clear the entire dictionary using clear().

Print the final result.

Add comments explaining the difference between pop() and popitem().
'''

Creating a dictionary with 5 key-value pairs
student = {
 "name": "Vaishnavi",
 "age": 18,
 "city": "Pune",
 "course": "AIML",
 "marks": 92
}
```

```
print("Original Dictionary:")
print(student)

Remove the last inserted key-value pair
removed_item1 = student.popitem()
print("\nFirst Removed Item:", removed_item1)

Remove another last inserted key-value pair
removed_item2 = student.popitem()
print("\nSecond Removed Item:", removed_item2)

Dictionary after popitem() operations
print("\nDictionary After popitem():")
print(student)

Clear the entire dictionary
student.clear()

Print the empty dictionary
print("\nDictionary After clear():")
print(student)
```

'''

Explanation in Comments

```
pop(key) removes a specific key and returns its value.
Example: student.pop("age")

popitem() removes and returns the last inserted
key-value pair as a tuple.

clear() removes all key-value pairs from the dictionary.

After clear(), the dictionary becomes empty.
'''
```

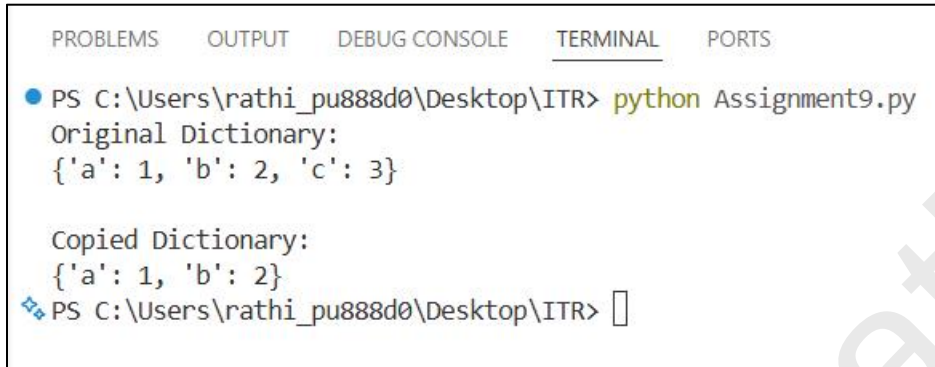
---

---

Q7. copy() and setdefault()

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Original Dictionary:
{'a': 1, 'b': 2, 'c': 3}

Copied Dictionary:
{'a': 1, 'b': 2}
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q7. (copy() and setdefault())

Create a dictionary:

```
d = {"a": 1, "b": 2}
```

Make a copy of it.

Use setdefault() to add key "c" with value 3 (if it doesn't exist).

Use setdefault() again for an existing key "a".

Print the original and copied dictionaries.

```
'''
```

```
Creating the dictionary
```

```
d = {"a": 1, "b": 2}
```

```
Making a copy of the dictionary
```

```
copied_d = d.copy()
```

```
Add key "c" with value 3 if it does not exist
```

```
d.setdefault("c", 3)
```

```
Try to add key "a" again
```

```
d.setdefault("a", 100)
```

```
Print the original dictionary
print("Original Dictionary:")
print(d)
```

```
Print the copied dictionary
print("\nCopied Dictionary:")
print(copied_d)
```

```
'''
```

Explanation in Comments

```
copy() creates a duplicate of the dictionary.
```

```
Changes made to the original dictionary
do not affect the copied dictionary.
```

```
setdefault(key, value) adds the key with the given value
only if the key does not already exist.
```

```
Here, "c" is added because it was not present.
```

```
"a" already exists, so its value remains 1
and is not changed to 100.
```

```
'''
```

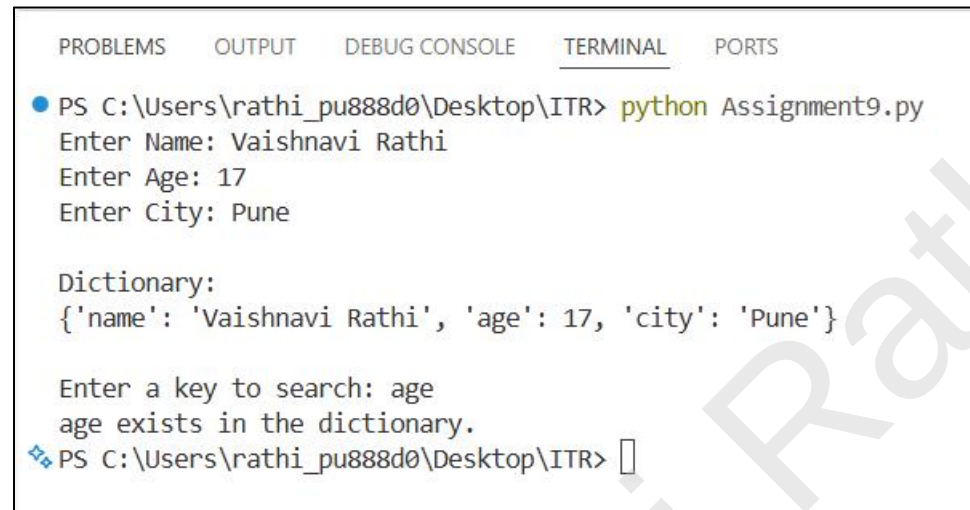
---

---

## Q8. fromkeys() and Membership

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Enter Name: Vaishnavi Rathi
Enter Age: 17
Enter City: Pune

Dictionary:
{'name': 'Vaishnavi Rathi', 'age': 17, 'city': 'Pune'}

Enter a key to search: age
age exists in the dictionary.
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 9 """

"""
Q8. (fromkeys() and Membership)

Write a program that:

Uses dict.fromkeys() to create a dictionary with keys
["name", "age", "city"] and default value None.

Takes user input to fill these keys.

Checks if a given key exists using in operator.
"""

Create a dictionary using fromkeys()
person = dict.fromkeys(["name", "age", "city"], None)

Take input from the user
person["name"] = input("Enter Name: ")
person["age"] = int(input("Enter Age: "))
person["city"] = input("Enter City: ")
```

```
Print the dictionary
print("\nDictionary:")
print(person)

Check whether a key exists
search_key = input("\nEnter a key to search: ")

if search_key in person:
 print(search_key, "exists in the dictionary.")
else:
 print(search_key, "does not exist in the dictionary.")

'''
```

Explanation in Comments

```
dict.fromkeys() creates a new dictionary using
the given keys and a default value.
```

```
Here, all keys initially have the value None.
```

```
Values are later updated using user input.
```

```
The 'in' operator checks whether a key
exists in the dictionary.
```

```
It returns True if the key is present,
otherwise it returns False.
```

```
'''
```

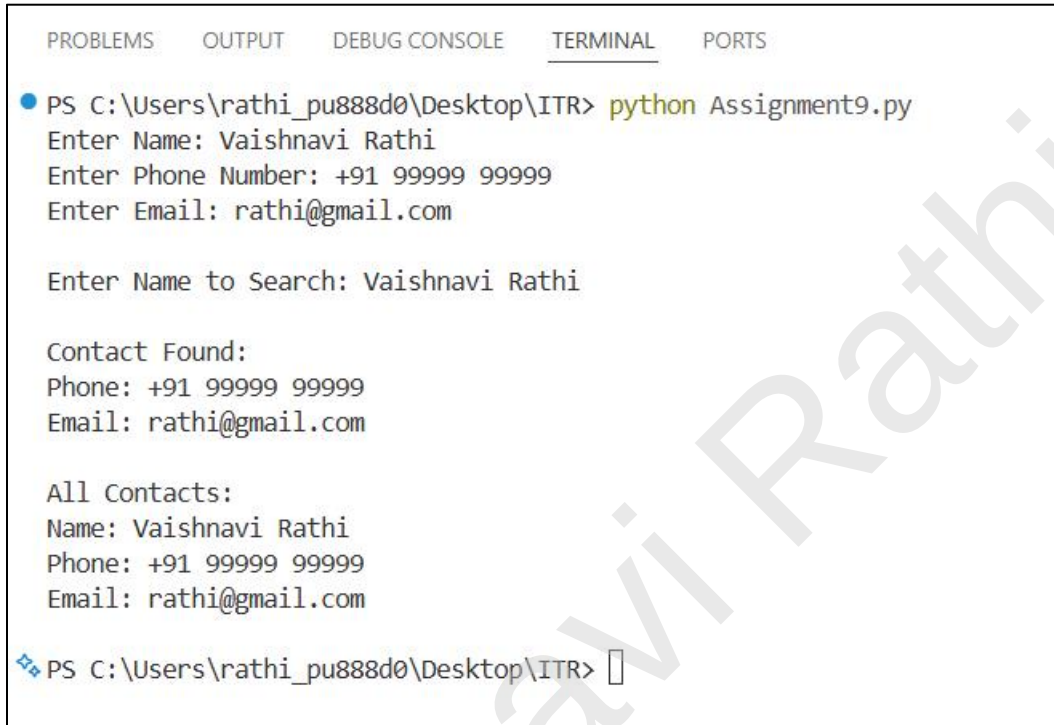
---

---

## Q9. Practical Application

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
Enter Name: Vaishnavi Rathi
Enter Phone Number: +91 99999 99999
Enter Email: rathi@gmail.com

Enter Name to Search: Vaishnavi Rathi

Contact Found:
Phone: +91 99999 99999
Email: rathi@gmail.com

All Contacts:
Name: Vaishnavi Rathi
Phone: +91 99999 99999
Email: rathi@gmail.com

PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

---

### Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q9. (Practical Application)

Write a program to store and manage contact information using a dictionary:

Take name, phone number, and email as input.  
Store them in a dictionary with name as key.  
Allow the user to search a contact by name using get().  
Print all contacts using items().

```
'''
```

```
Create an empty dictionary for contacts
contacts = {}
```

```
Take contact details from the user
name = input("Enter Name: ")
```

```

phone = input("Enter Phone Number: ")
email = input("Enter Email: ")

Store contact information
contacts[name] = {
 "phone": phone,
 "email": email
}

Search for a contact using get()
search_name = input("\nEnter Name to Search: ")

contact = contacts.get(search_name)

if contact:
 print("\nContact Found:")
 print("Phone:", contact["phone"])
 print("Email:", contact["email"])
else:
 print("Contact Not Found")

Print all contacts using items()
print("\nAll Contacts:")

for name, details in contacts.items():
 print("Name:", name)
 print("Phone:", details["phone"])
 print("Email:", details["email"])
 print()

'''
Explanation in Comments

The contact name is used as the key in the dictionary.

Each contact stores another dictionary containing
phone number and email.

get() is used to search for a contact safely.
It returns None if the key is not found.

items() returns all key-value pairs from the dictionary.

A for loop is used to display all contacts.
'''

```

---

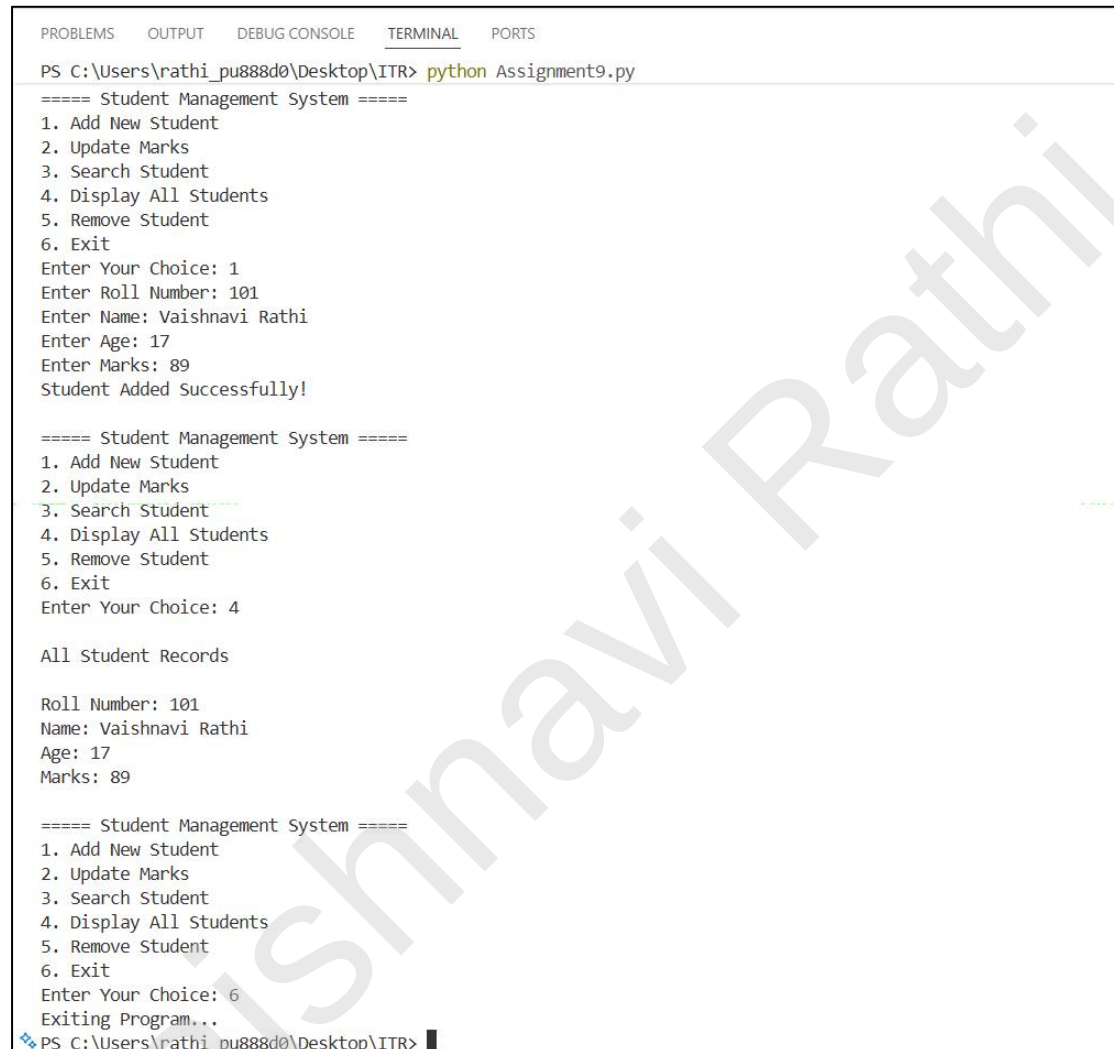
--

---

## Q10. Mini Project - Combined Concepts

---

### Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR> python Assignment9.py
===== Student Management System =====
1. Add New Student
2. Update Marks
3. Search Student
4. Display All Students
5. Remove Student
6. Exit
Enter Your Choice: 1
Enter Roll Number: 101
Enter Name: Vaishnavi Rath
Enter Age: 17
Enter Marks: 89
Student Added Successfully!

===== Student Management System =====
1. Add New Student
2. Update Marks
3. Search Student
4. Display All Students
5. Remove Student
6. Exit
Enter Your Choice: 4

All Student Records

Roll Number: 101
Name: Vaishnavi Rath
Age: 17
Marks: 89

===== Student Management System =====
1. Add New Student
2. Update Marks
3. Search Student
4. Display All Students
5. Remove Student
6. Exit
Enter Your Choice: 6
Exiting Program...
PS C:\Users\rathi_pu888d0\Desktop\ITR>
```

### Program Code:

```
""" ASSIGNMENT 9 """
```

```
'''
```

Q10. (Mini Project - Combined Concepts)

Create a Student Management System using a dictionary.

Menu:

1. Add New Student
2. Update Marks of a Student
3. Search Student by Roll Number
4. Display All Students
5. Remove a Student
6. Exit

```
'''
```

```
Dictionary to store student records
```

```
students = {}
```

```
Run the menu until the user chooses Exit
```

```
while True:
```

```
 print("\n===== Student Management System =====")
 print("1. Add New Student")
 print("2. Update Marks")
 print("3. Search Student")
 print("4. Display All Students")
 print("5. Remove Student")
 print("6. Exit")
```

```
 choice = int(input("Enter Your Choice: "))
```

```
Add a new student
```

```
if choice == 1:
```

```
 roll_no = int(input("Enter Roll Number: "))
 name = input("Enter Name: ")
 age = int(input("Enter Age: "))
 marks = int(input("Enter Marks: "))
```

```
 students[roll_no] = {
 "name": name,
 "age": age,
 "marks": marks
 }
```

```
 print("Student Added Successfully!")
```

```
Update marks of an existing student
```

```

elif choice == 2:
 roll_no = int(input("Enter Roll Number: "))

 student = students.get(roll_no)

 if student:
 new_marks = int(input("Enter New Marks: "))

 # update() modifies the existing dictionary
 student.update({"marks": new_marks})

 print("Marks Updated Successfully!")
 else:
 print("Student Not Found!")

Search student by roll number
elif choice == 3:
 roll_no = int(input("Enter Roll Number: "))

 student = students.get(roll_no)

 if student:
 print("\nStudent Details")
 print("Name :", student["name"])
 print("Age :", student["age"])
 print("Marks :", student["marks"])
 else:
 print("Student Not Found!")

Display all students
elif choice == 4:

 if len(students) == 0:
 print("No Student Records Available.")
 else:
 print("\nAll Student Records")

 for roll_no, details in students.items():
 print("\nRoll Number:", roll_no)
 print("Name:", details["name"])
 print("Age:", details["age"])
 print("Marks:", details["marks"])

Remove a student
elif choice == 5:
 roll_no = int(input("Enter Roll Number: "))

 removed_student = students.pop(roll_no, None)

 if removed_student:

```

```

 print("Student Removed Successfully!")
 else:
 print("Student Not Found!")

 # Exit the program
 elif choice == 6:
 print("Exiting Program...")
 break

 # Handle invalid choices
 else:
 print("Invalid Choice! Please Try Again.")

'''
Explanation in Comments

students is a nested dictionary where:
Roll Number = Key
Student Details = Value

get() is used to search for a student safely.
It returns None if the roll number is not found.

update() is used to modify the marks of a student.

items() is used to display all student records.

pop() removes a student record using roll number.

The while loop keeps displaying the menu
until the user chooses Exit.
'''

```

---